DOCUMENTATION Utilisateurs

Logiciel : MiBombo v1.0

Analyseur de Trames SNMP Sécurisé

Version 1.1.0 implémenté le 16 **Janvier 2026**

Équipe de Développement

Rôle	Nom
Développeur	M. Dapra Enzo
Développeur	M. Boudina Salah

Encadrement Client

Rôle	Nom
Client (et enseignant)	M. Druon Sebastien
Client (et enseignant)	M. Borelly Cristophe
Client (et enseignant)	M. Comby Frederic

IUT de Béziers – BUT Réseaux & Télécommunications

SAE 501/502 – Projet Logiciel Sniffer SNMP par groupe ENSA

Table des Matières

1. Introduction
   - 1.1 Présentation du Projet
   - 1.2 Objectifs
   - 1.3 Public Cible
   - 1.4 Conventions Documentaires
2. Architecture du Système
   - 2.1 Vue d'Ensemble
   - 2.2 Composants Principaux
   - 2.3 Technologies Utilisées
   - 2.4 Flux de Données
3. Prérequis et Environnement
   - 3.1 Configuration Matérielle
   - 3.2 Configuration Logicielle
   - 3.3 Dépendances Python
   - 3.4 Ports Réseau Utilisés
4. Installation et Déploiement
   - 4.1 Récupération depuis GitHub
   - 4.2 Installation Manuelle (Développeur)
   - 4.3 Installation via Paquet Debian
   - 4.4 Déploiement Docker
   - 4.5 Vérification de l'Installation
5. Configuration
   - 5.1 Variables d'Environnement (.env)
   - 5.2 Infrastructure PKI
   - 5.3 Configuration Base de Données
   - 5.4 Configuration Réseau
6. Guide d'Utilisation
   - 6.1 Lancement de l'Application
   - 6.2 Authentification
   - 6.3 Interface Principale (GUI)
   - 6.4 API REST
   - 6.5 Mode Ligne de Commande (CLI)
7. Fonctionnalités Détaillées
   - 7.1 Capture de Paquets SNMP
   - 7.2 Analyse et Déchiffrement SNMPv3
   - 7.3 Détection d'Anomalies

- 7.4 Gestion des Utilisateurs
- 7.5 Topologie Réseau
- 7.6 Export et Rapports

8. Sécurité
    - 8.1 Authentification Sécurisée
    - 8.2 Gestion des Certificats
    - 8.3 Chiffrement des Données
    - 8.4 Bonnes Pratiques
9. Maintenance et Dépannage
    - 9.1 Logs et Diagnostics
    - 9.2 Problèmes Courants
    - 9.3 Sauvegarde et Restauration
    - 9.4 Mises à Jour
10. Annexes
    - 10.1 Liste des Figures
    - 10.2 Arborescence du Projet
    - 10.3 Glossaire
    - 10.4 Références
11. Perspectives d'Évolution
    - 11.1 Roadmap Technique
    - 11.2 Améliorations Planifiées
    - 11.3 Contributions et Retours

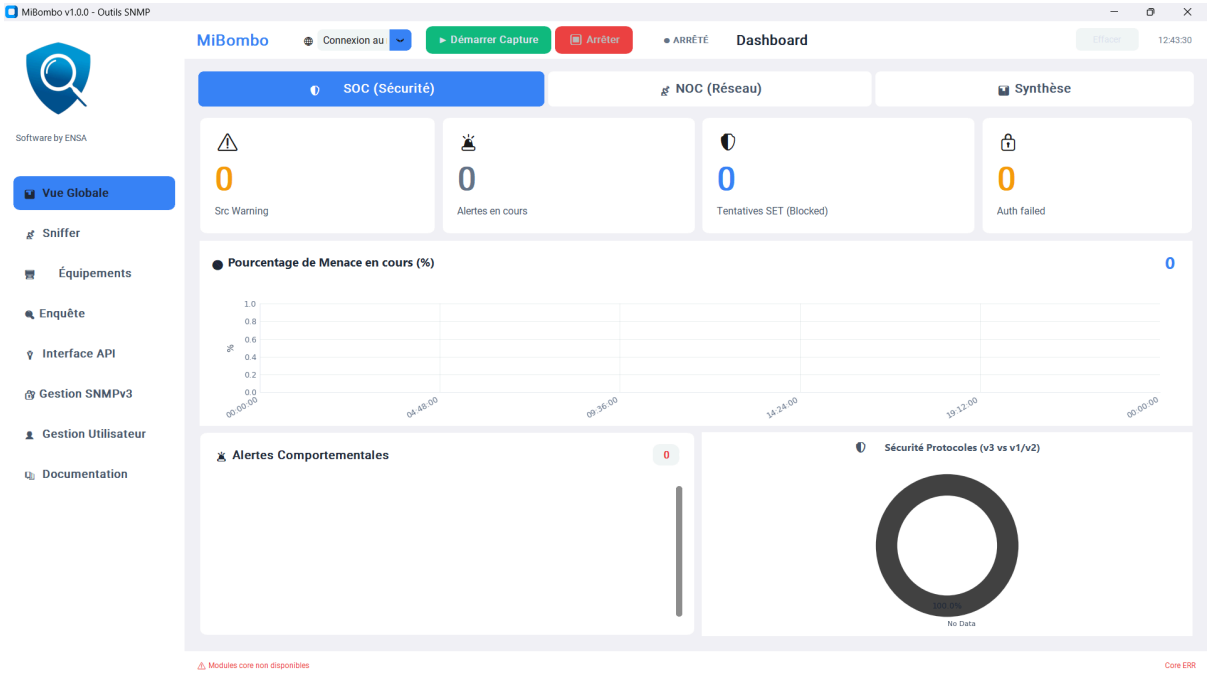
1. Introduction

1.1 Présentation du Projet

MiBombo est une solution logicielle complète de gestion et d'analyse de trames SNMP (Simple Network Management Protocol). Développée dans le cadre de la SAE 501/502 du BUT Réseaux & Télécommunications pour vous , cette application répond aux besoins des administrateurs réseau en matière de supervision et de sécurité.

Le protocole SNMP est omniprésent dans les infrastructures réseau modernes. Il permet de surveiller et de gérer les équipements (routeurs, switches, serveurs, imprimantes, etc.) de manière centralisée. Cependant, l'analyse du trafic SNMP peut s'avérer complexe, notamment avec l'introduction de SNMPv3 et ses mécanismes de chiffrement.

MiBombo offre une interface unifiée pour : - **Capturer** les trames SNMP en temps réel sur le réseau - **Analyser** le contenu des paquets (v1, v2c et v3) - **Déchiffrer** les messages SNMPv3 avec authentification et confidentialité - **Détecter** les anomalies comportementales sur le réseau - **Alerter** les opérateurs en cas de comportement suspect



cf. Figure 1 : Interface principale de MiBombo

1.2 Objectifs

Les objectifs principaux du projet sont :

Objectif	Description
Technique	Développement d'un outil capable de capturer, analyser et stocker les trames SNMP v1/v2c/v3
Sécurité	Implémentation d'un système d'authentification sécurisé
Ergonomie	Interface graphique intuitive pour les utilisateurs
Extensibilité	API REST documentée permettant l'intégration avec d'autres outils
Autonomie	Déploiement simplifié via paquet Debian ou conteneur Docker

1.3 Public Cible

Cette documentation s'adresse aux profils/utilisateurs suivants :

- **Administrateurs Système** : Installation, configuration et maintenance
- **Ingénieurs Réseau** : Utilisation quotidienne pour la supervision
- **Analystes Sécurité (SOC)** : Détection d'anomalies et investigation
- **Développeurs** : Intégration API et contribution au code
- **Ou autre aventuriers** qui souhaites utiliser notre software

1.4 Conventions Documentaires

Ce document utilise les conventions suivantes pour faciliter la lecture :

Convention	Signification
`commande`	Commande à exécuter dans un terminal
Terme	Terme important ou nom de menu
cf. Figure X	Référence à une capture d'écran à insérer
⚠️	Avertissement important
💡	Conseil ou astuce
📁	Chemin de fichier ou répertoire

Codes Couleur des Alertes

Dans l'interface MiBombo, les alertes utilisent un système de couleurs standardisé :

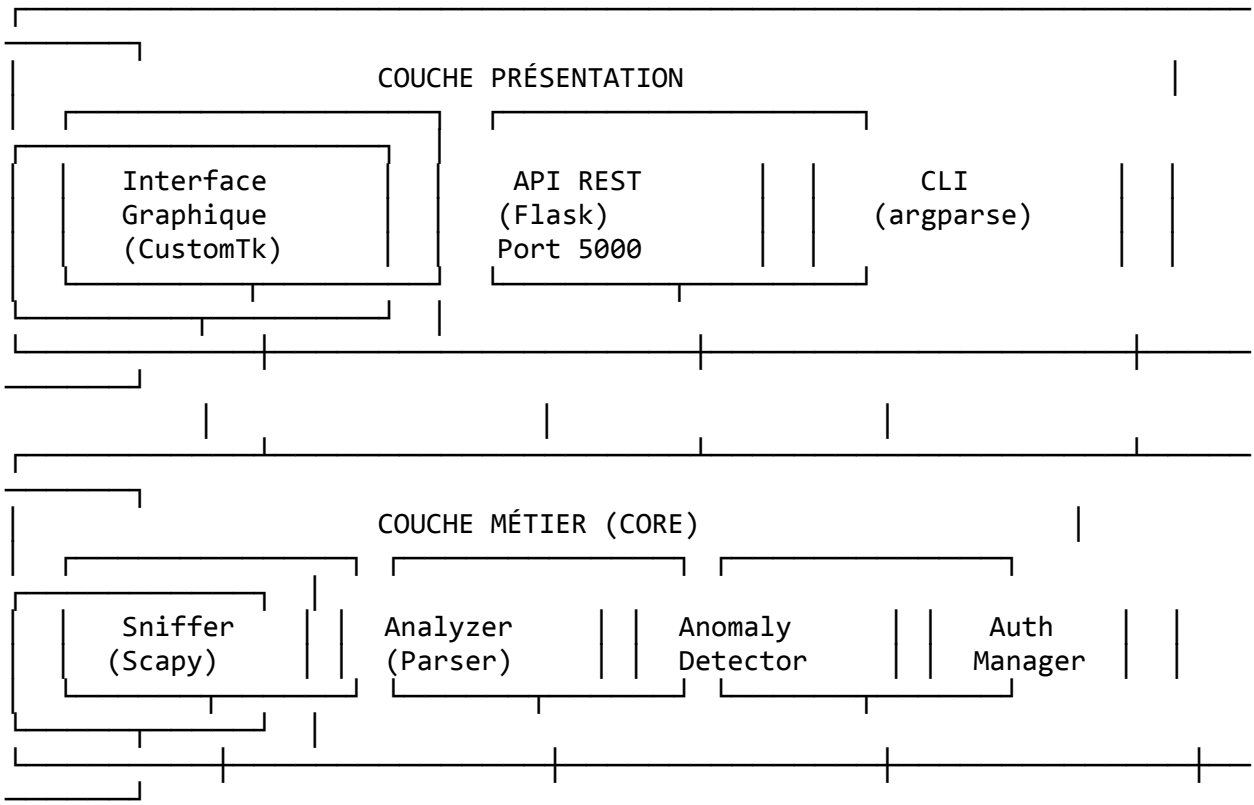
Couleur	Niveau	Signification
🟢 Vert	OK / Info	Fonctionnement normal, information
🟡 Jaune	Warning	Attention requise, anomalie mineure
🟠 Orange	Critical	Situation critique nécessitant une action
🔴 Rouge	Emergency	Urgence, intervention immédiate requise

---

2. Architecture du Système

2.1 Vue d'Ensemble

MiBombo adopte une architecture mi modulaire mi monolith composée de plusieurs couches distinctes :



```
          │              │              │              │
┌─────────┴──────────────┴──────────────┴──────────────┴──────────────────┴──
────┐
│                        COUCHE DONNÉES                                    │
│   ┌──────────────────────────┐                                           │
┌──────────────────────────────────────────┐   │
│   │   PostgreSQL          │  │   Fichiers (PCAP, Config, Logs)    │   │
│   └──────────────────────────┘                                           │
└──────────────────────────────────────────┘   │
└──────────────────────────────────────────────────────────────────────────
────┘
```

cf. Figure 2 : Schéma d'architecture de MiBombo

2.2 Composants Principaux

2.2.1 Module Sniffer (`core/sniffer.py`)

Responsable de la capture des paquets réseau en temps réel grâce à la bibliothèque **Scapy**. Il écoute sur l'interface réseau spécifiée et filtre les trames SNMP (ports UDP 161 et 162).

Caractéristiques : - Capture en mode promiscuous - Filtre BPF configurable - File d'attente thread-safe pour le traitement asynchrone

2.2.2 Module Analyzer (`core/analyzer.py`)

Parse et interprète les paquets SNMP capturés. Supporte les trois versions du protocole :

Version	Fonctionnalités
v1	Parsing complet, extraction des OID et valeurs
v2c	Idem v1 + support des types étendus
v3	Authentification (MD5, SHA), Chiffrement (DES, AES), USM

2.2.3 Module Anomaly Detector (`core/anomaly_detector.py`)

Système de détection comportementale basé sur des règles et des seuils statistiques :

- Détection de scan de communautés
- Alerte sur trafic anormal (pics d'activité)
- Identification de sources suspectes
- Classification par sévérité (Info, Warning, Critical, Emergency)

2.2.4 Module Authentication (`core/secure_authentication.py`)

Gestion de l'authentification et des autorisations :

- Hashage des mots de passe (Argon2/bcrypt)
- Authentification à deux facteurs (2FA) par e-mail
- Gestion des rôles (admin, operator, viewer)
- Sessions sécurisées avec tokens JWT

2.3 Technologies Utilisées

Catégorie	Technologie	Version
Langage	Python	3.11+
GUI	CustomTkinter	5.x
Capture Réseau	Scapy	2.5+
API Web	Flask + Flask-CORS	3.x
Base de Données	PostgreSQL / SQLite	16 / 3
Chiffrement	Cryptography	41+
Graphiques	Matplotlib	3.8+
Conteneurisation	Docker + Compose	24+

---

3. Prérequis et Environnement

3.1 Configuration Matérielle

Ressource	Minimum	Recommandé
Processeur	Dual-core 2 GHz	Quad-core 3 GHz
Mémoire RAM	2 Go	4 Go ou plus
Stockage	500 Mo	2 Go (pour les captures)
Réseau	1 interface Ethernet	Interface dédiée au mirroring

Note : La capture de paquets nécessite des privilèges élevés (root/sudo) ou des capabilities réseau appropriées.

3.2 Configuration Logicielle

Système d'Exploitation

MiBombo est conçu pour fonctionner sur les systèmes Linux (et en partie sur Windows) :

Distribution	Version Testée	Statut
Debian	12 (Bookworm), 13 (Trixie)	✅ Supporté
Ubuntu	22.04 LTS, 24.04 LTS	✅ Supporté
Rocky Linux	9.x	⚠️ Partiel
Windows	10/11 (WSL2)	⚠️ Expérimental

Paquets Système Requis

```
# Debian/Ubuntu
sudo apt update
sudo apt install -y python3 python3-pip python3-venv \
    libpq-dev postgresql-client \
    tcpdump wireshark-common \
    openssl git curl
```

3.3 Dépendances Python

Les dépendances Python sont listées dans le fichier `requirements.txt` :

```
# Framework Web
Flask>=3.0.0
Flask-CORS>=4.0.0
flask-socketio>=5.3.0

# Capture Réseau
scapy>=2.5.0

# Interface Graphique
customtkinter>=5.2.0
Pillow>=10.0.0
matplotlib>=3.8.0

# Base de Données
psycopg2-binary>=2.9.0
influxdb-client>=1.36.0

# Sécurité
cryptography>=41.0.0
pyotp>=2.9.0
argon2-cffi>=23.1.0

# Utilitaires
python-dotenv>=1.0.0
```

```
requests>=2.31.0
psutil>=5.9.0
```

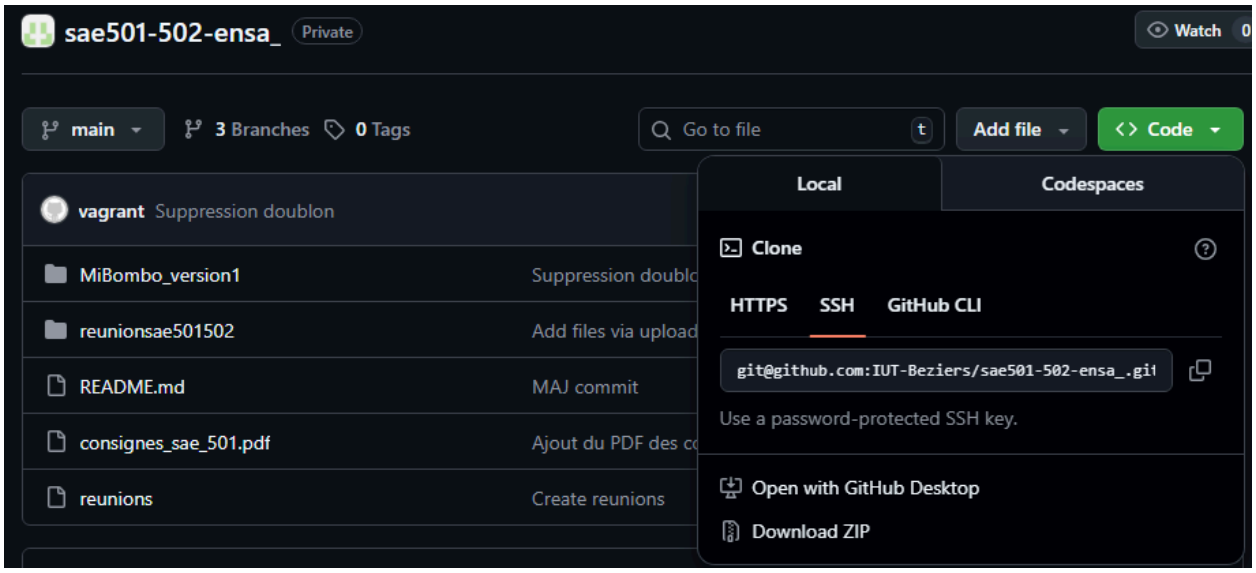
---

4. Installation et Déploiement

4.1 Récupération depuis GitHub

Étape 1 : Cloner le Dépôt

```
# Clonage du dépôt (accès public)
git clone https://github.com/IUT-Beziers/sae501-502-ensa_.git

# OU avec un token d'accès personnel (accès privé)
git clone https://<VOTRE_TOKEN>@github.com/IUT-Beziers/sae501-502-ensa_.git
```



cf. Figure 4 : Clonage du dépôt GitHub

Étape 2 : Naviguer vers le Répertoire

```
cd sae501-502-ensa_/MiBombo_version1
```

Étape 3 : Vérifier la Structure

```
ls -la
```

Vous devriez voir la structure suivante :

```
MiBombo_version1/
├── api/                 # API REST Flask
├── assets/              # Ressources graphiques (logo, icônes)
├── config/              # Fichiers de configuration (générés)
├── core/                # Modules métier principaux
├── data/                # Données et logs (générés)
├── debian/              # Scripts de packaging Debian
├── docs/                # Documentation additionnelle
├── gui/                 # Interface graphique
├── pki/                 # Certificats PKI (générés)
├── scripts/             # Scripts utilitaires
├── tests/               # Tests unitaires et d'intégration
├── utils/               # Utilitaires divers
├── .env.example         # Template des variables d'environnement
├── main.py              # Point d'entrée principal
├── requirements.txt     # Dépendances Python
└── run.sh               # Script de lancement
```

cf. Figure 5 : Arborescence du projet MiBombo

4.2 Installation Manuelle (Développeur)

Cette méthode est recommandée pour les développeurs ou pour un déploiement de test.

Étape 1 : Créer un Environnement Virtuel

```
python3 -m venv venv
source venv/bin/activate
```

cf. Figure 6 : Création et activation du virtual environment

Étape 2 : Installer les Dépendances

```
pip install --upgrade pip
pip install -r requirements.txt
```

cf. Figure 7 : Installation des dépendances Python

Étape 3 : Configurer les Variables d'Environnement

```
cp .env.example .env
nano .env
```

Modifiez les valeurs par défaut, notamment les clés de sécurité :

```
# Génération d'une clé sécurisée
python3 -c "import secrets; print(secrets.token_urlsafe(32))"
```

cf. Figure 8 : Configuration du fichier .env

Étape 4 : Générer l'Infrastructure PKI

```
chmod +x scripts/pki/generate_pki.sh
./scripts/pki/generate_pki.sh
```

Cette commande génère : - Autorité de certification racine (Root CA) - Autorité de certification subordonnée (Sub CA) - Certificat de la station MiBombo

cf. Figure 9 : Génération des certificats PKI

Étape 5 : Lancer l'Application

```
sudo ./run.sh
```

Ou directement :

```
sudo ./venv/bin/python main.py
```

cf. Figure 10 : Démarrage de MiBombo

4.3 Installation via Paquet Debian

Cette méthode est recommandée pour un déploiement en production.

Étape 1 : Construire le Paquet

```
chmod +x build_deb.sh
./build_deb.sh
```

cf. Figure 11 : Construction du paquet .deb

Étape 2 : Installer le Paquet

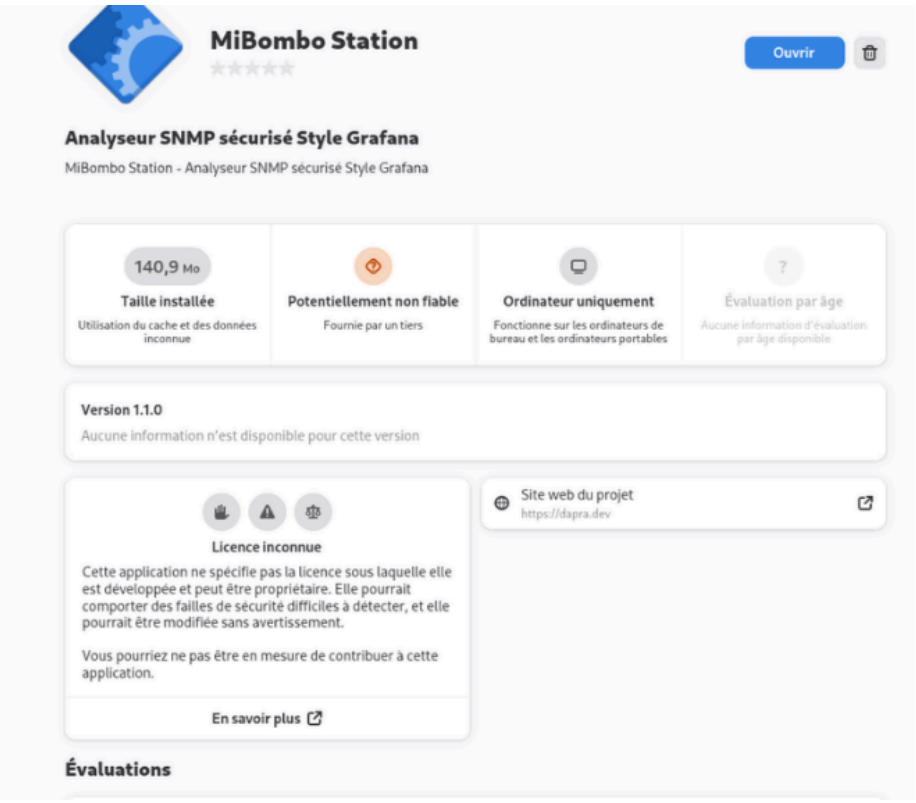
```
sudo dpkg -i mibombo_1.1.0_all.deb

# Résoudre les dépendances manquantes si nécessaire
sudo apt-get install -f
```

cf. Figure 12 : Installation du paquet Debian

Étape 3 : Lancer l'Application

```
sudo mibombo
```

L'application est également accessible depuis le menu des applications du bureau.



cf. Figure 13 : Lancement via le menu Applications

4.4 Déploiement Docker

Étape 1 : Construire l'Image

```
docker-compose build
```

Étape 2 : Démarrer les Conteneurs

```
docker-compose up -d
```

La commande ci dessus démarre : - Le conteneur MiBombo (application principale)

Étape 3 : Vérifier l'État du déploiement

```
docker-compose ps
docker-compose logs -f mibombo-app
```

Étape 4 : Accéder à l'Application

- **API REST** : `https://localhost:5000`
- **Documentation API** : `https://localhost:5000/api/docs`

5. Configuration

5.1 Variables d'Environnement (.env)

Le fichier `.env` centralise toutes les configurations sensibles. Il ne doit **jamais** être commité sur Git.

Variable	Description	Exemple
`FLASK_SECRET_KEY`	Clé secrète Flask pour les sessions	`votre_cle_aleatoire_32_chars`
`SNIFFER_KEY`	Clé de chiffrement des captures	`Fernet.generate_key()`
`AUTH_KEY`	Clé de chiffrement des tokens auth	`Fernet.generate_key()`
`ENCRYPTION_KEY`	Clé générale de chiffrement	`Fernet.generate_key()`
`DATABASE_URL`	URL de connexion PostgreSQL	`postgresql://user:pass@host/db`
`INFLUXDB_TOKEN`	Token d'accès InfluxDB	`my-super-secret-token`
`INFLUXDB_ORG`	Organisation InfluxDB	`mibombo-org`

Variable	Description	Exemple
`INFLUXDB_BUCKET`	Bucket InfluxDB	`mibombo-bucket`

Génération Automatique des Clés

Le script `postinst` du paquet Debian génère automatiquement ces clés lors de l'installation. Si vous souhaitez faire une installation manuelle :

```
python3 -c "
from cryptography.fernet import Fernet
print('SNIFFER_KEY=' + Fernet.generate_key().decode())
print('AUTH_KEY=' + Fernet.generate_key().decode())
print('ENCRYPTION_KEY=' + Fernet.generate_key().decode())
"
```

5.2 Infrastructure PKI

MiBombo utilise une infrastructure à clé publique (PKI) hiérarchique pour sécuriser les communications HTTPS. Préférable pour l'utilisation de notre logiciel dans votre SI.

Structure de la PKI

```
pki/
├── root_ca/
│   ├── rootCA.key          # Clé privée Root CA (protéger absolument)
│   ├── rootCA.pem          # Certificat Root CA
│   └── rootCA.srl          # Numéro de série
├── sub_ca/
│   ├── subCA.key           # Clé privée Sub CA
│   ├── subCA.pem           # Certificat Sub CA
│   └── subCA.csr           # Requête de signature
└── station/
    ├── station.key         # Clé privée Station
    ├── station.pem         # Certificat Station
    └── station.csr         # Requête de signature
```

Durées de Validité

Certificat	Durée	Commande de Renouvellement
Root CA	5 ans	Regénération complète
Sub CA	1 an	`./scripts/pki/generate_pki.sh`
Station	90 jours	`./scripts/pki/renew_station_cert.sh`

cf. Figure 16 : Hiérarchie des certificats PKI

NB : Peut être modifié par nous ou par vous pour changer cette dite periode

5.3 Configuration Base de Données

SQLite (Par défaut)

MiBombo utilise SQLite par défaut pour une installation simplifiée. Le fichier `mibombo.db` est créé automatiquement au premier lancement.

PostgreSQL (Production)

Pour un environnement de production, PostgreSQL est fortement recommandé :

```
# Création de la base de données
sudo -u postgres psql
CREATE DATABASE mibombo;
CREATE USER mibombo WITH ENCRYPTED PASSWORD 'votre_mot_de_passe';
GRANT ALL PRIVILEGES ON DATABASE mibombo TO mibombo;
```

Configurez ensuite `DATABASE_URL` dans `.env` :

```
DATABASE_URL=postgresql://mibombo:votre_mot_de_passe@ipadresse/mibombo
```

---

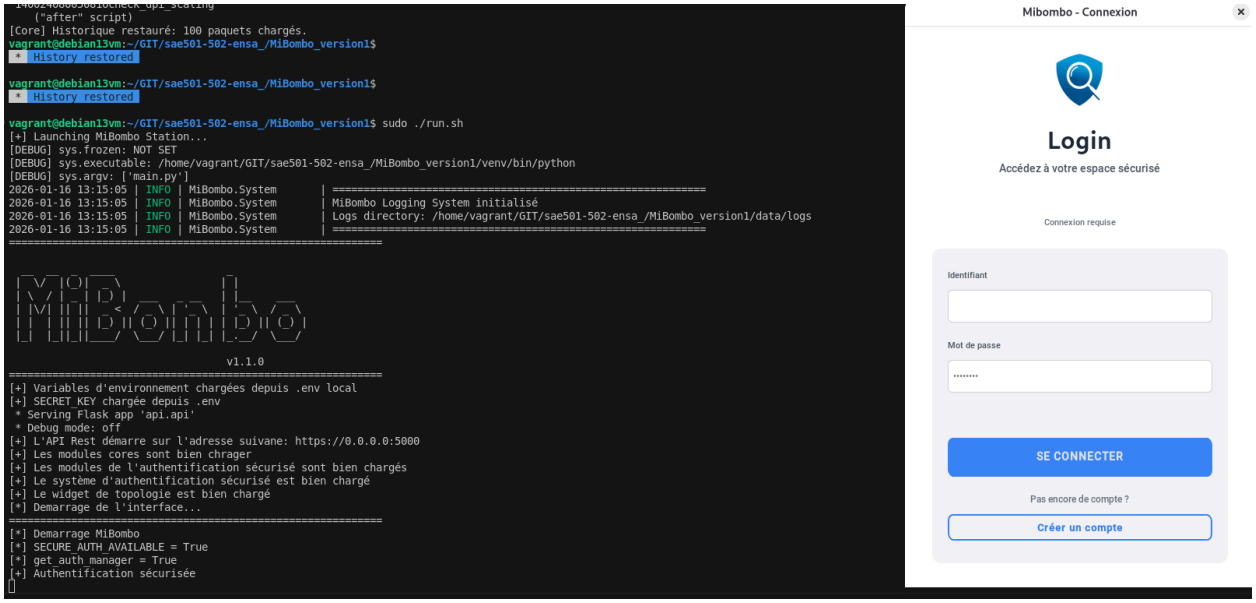
6. Guide d'Utilisation

6.1 Lancement de l'Application

Mode Graphique (Défaut)

```
sudo ./run.sh
# ou
sudo mibombo
```

Au démarrage, l'application affiche : 1. Le logo ASCII MiBombo 2. Les modules chargés 3. L'URL de l'API REST 4. La fenêtre de connexion



cf. Figure 17 : Écran de démarrage de l'application

Mode CLI (Ligne de Commande)

```
sudo python main.py --cli -i eth0
```

Options disponibles :

Option	Description	Défaut
`-i, --interface`	Interface réseau à écouter	`eth0`
`-f, --filter`	Filtre BPF personnalisé	`udp port 161 or 162`
`-d, --database`	Chemin vers la base SQLite	`mibombo.db`
`--with-api`	Active l'API REST en arrière-plan	Désactivé

cf. Figure 18 : Utilisation en mode CLI

Mode API Seule

```
sudo python main.py --api-only --api-port 5000
```

Ce mode lance uniquement le serveur API REST, sans interface graphique.

⚠️ ATTENTION : Ce mod est encore en version instable il est préférable d'utiliser le GUI

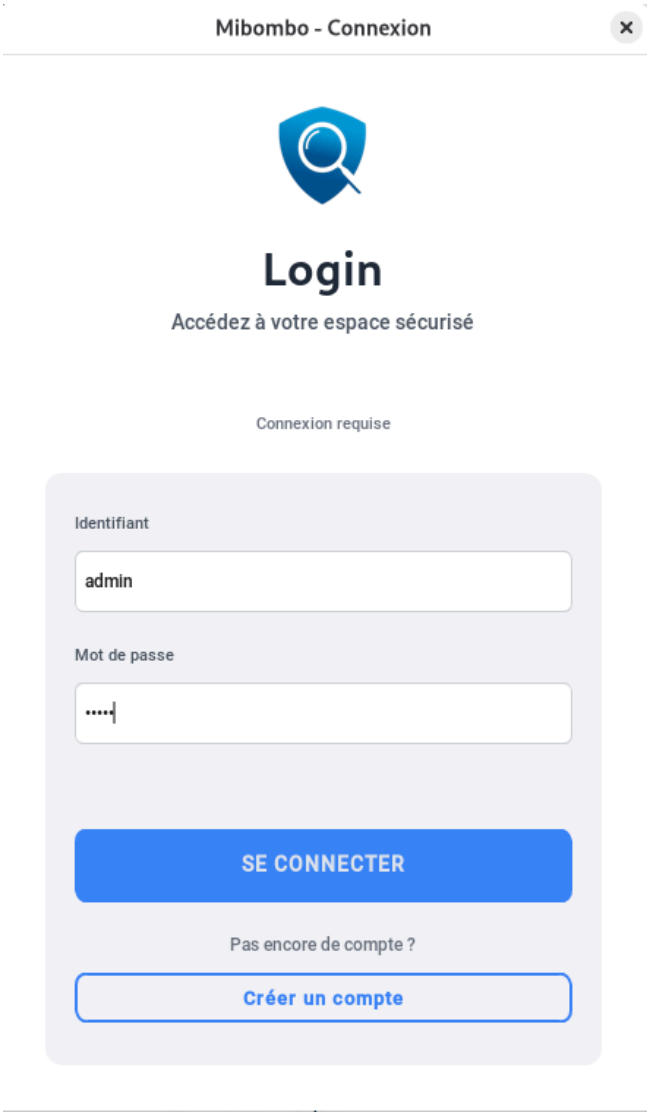
6.2 Authentification

Première Connexion

Lors du premier lancement, un compte administrateur par défaut est créé :

Champ	Valeur
Identifiant	`admin`
Mot de passe	`admin`

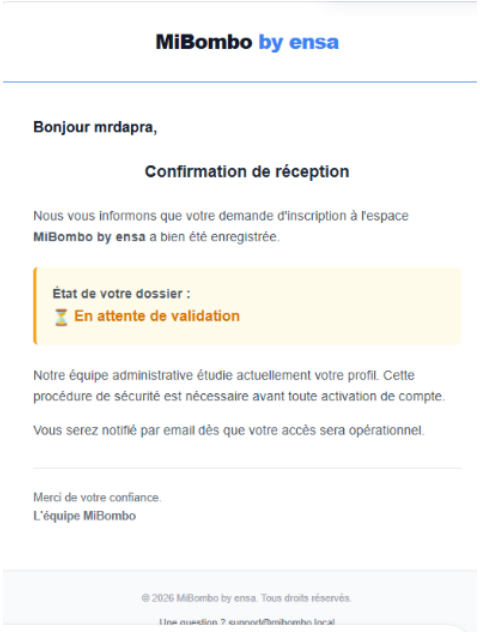
⚠️ **ATTENTION** : Changez immédiatement ce mot de passe après la première connexion !



cf. Figure 19 : Écran de connexion

Authentification Par Mail (SMTP)

En activant , l'utilisateur reçoit par e-mail une notification de demande après avoir entré son mot de passe.



MiBombo by ensa

Bonjour mrdapra,

Confirmation de réception

Nous vous informons que votre demande d'inscription à l'espace **MiBombo by ensa** a bien été enregistrée.

État de votre dossier :
⏳ **En attente de validation**

Notre équipe administrative étudie actuellement votre profil. Cette procédure de sécurité est nécessaire avant toute activation de compte.

Vous serez notifié par email dès que votre accès sera opérationnel.

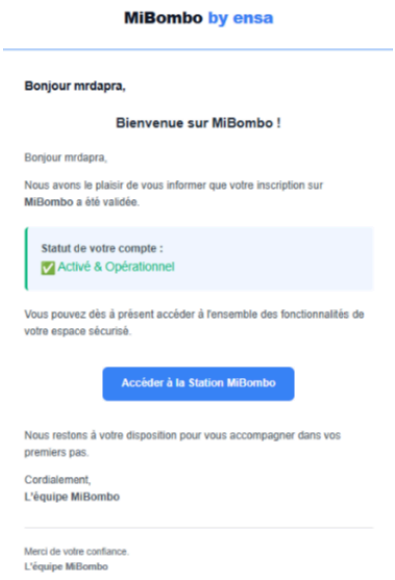
Merci de votre confiance.
L'équipe MiBombo

© 2026 MiBombo by ensa. Tous droits réservés.

cf. Figure 20 : Mail confirmation de demande

Création de Compte

Les nouveaux utilisateurs peuvent demander la création d'un compte : 1. Cliquer sur "Créer un compte" 2. Remplir le formulaire (identifiant, e-mail, mot de passe) 3. La demande est envoyée à l'administrateur pour validation



MiBombo by ensa

Bonjour mrdapra,

Bienvenue sur MiBombo !

Bonjour mrdapra,

Nous avons le plaisir de vous informer que votre inscription sur MiBombo a été validée.

Statut de votre compte :
✅ Activé & Opérationnel

Vous pouvez dès à présent accéder à l'ensemble des fonctionnalités de votre espace sécurisé.

Accéder à la Station MiBombo

Nous restons à votre disposition pour vous accompagner dans vos premiers pas.

Cordialement,
L'équipe MiBombo

Merci de votre confiance.
L'équipe MiBombo

cf. Figure 21 : Formulaire validation inscription

6.3 Interface Principale (GUI)

Vue d'Ensemble

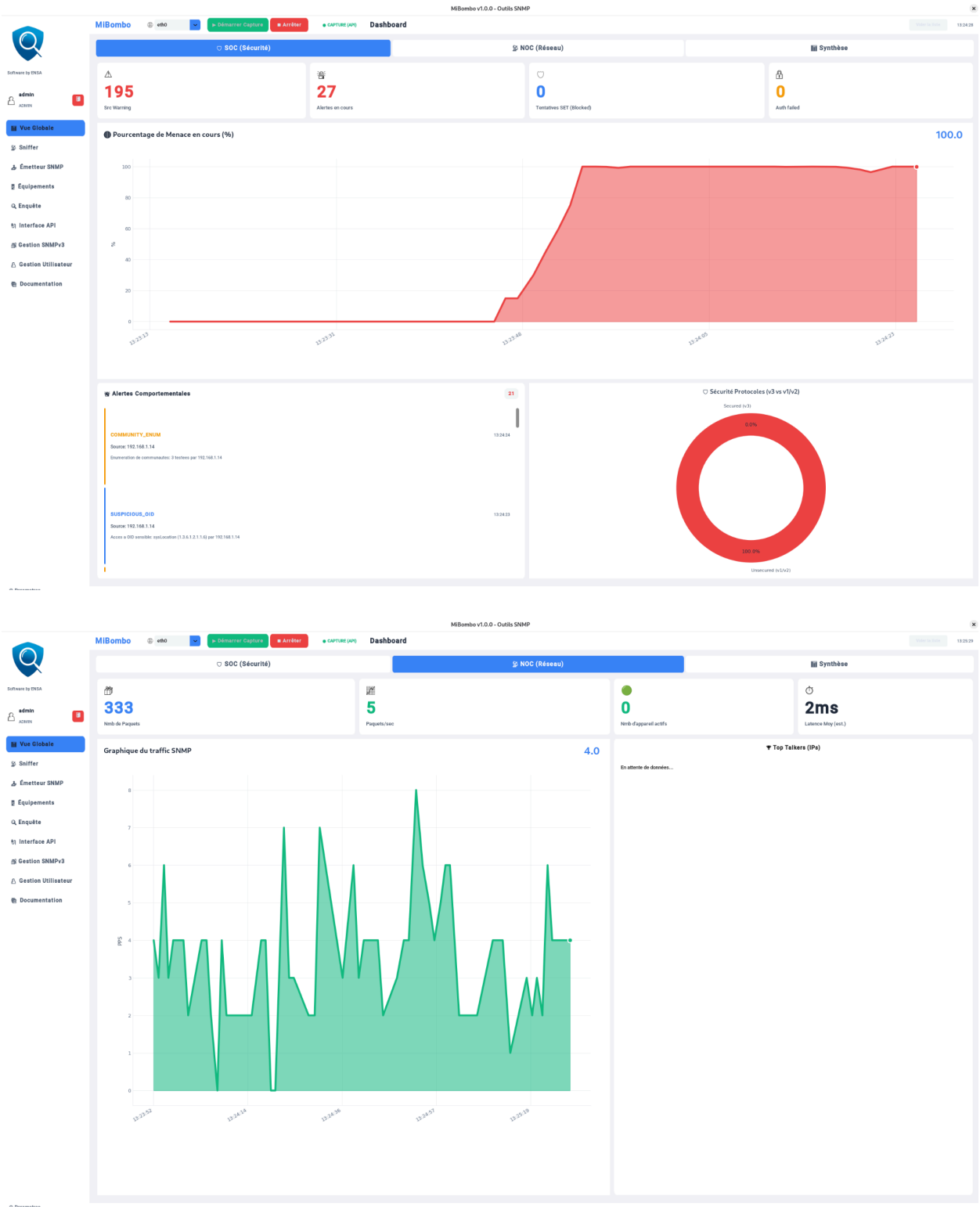
L'interface principale est organisée en plusieurs onglets :

Onglet	Description
SOC (Security)	Tableau de bord sécurité avec alertes et anomalies
NOC (Network)	Statistiques réseau et performances
Synthèse	Vue consolidée des captures SNMP
Historique	Recherche et filtrage des paquets passés
Topologie	Cartographie du réseau découvert
API	Documentation interactive de l'API
Paramètres	Configuration de l'application

cf. Figure 22 : Organisation Interface principale avec onglets

Onglet SOC

Cet onglet présente : - **Jauges temps réel** : CPU, RAM, débit réseau - **Graphique temporel** : Évolution du nombre de paquets/seconde - **Panneau d'alertes** : Liste des anomalies détectées avec sévérité - **Top IP** : Classement des hôtes les plus actifs



cf. Figure 23 : Vue détaillée du dashboard

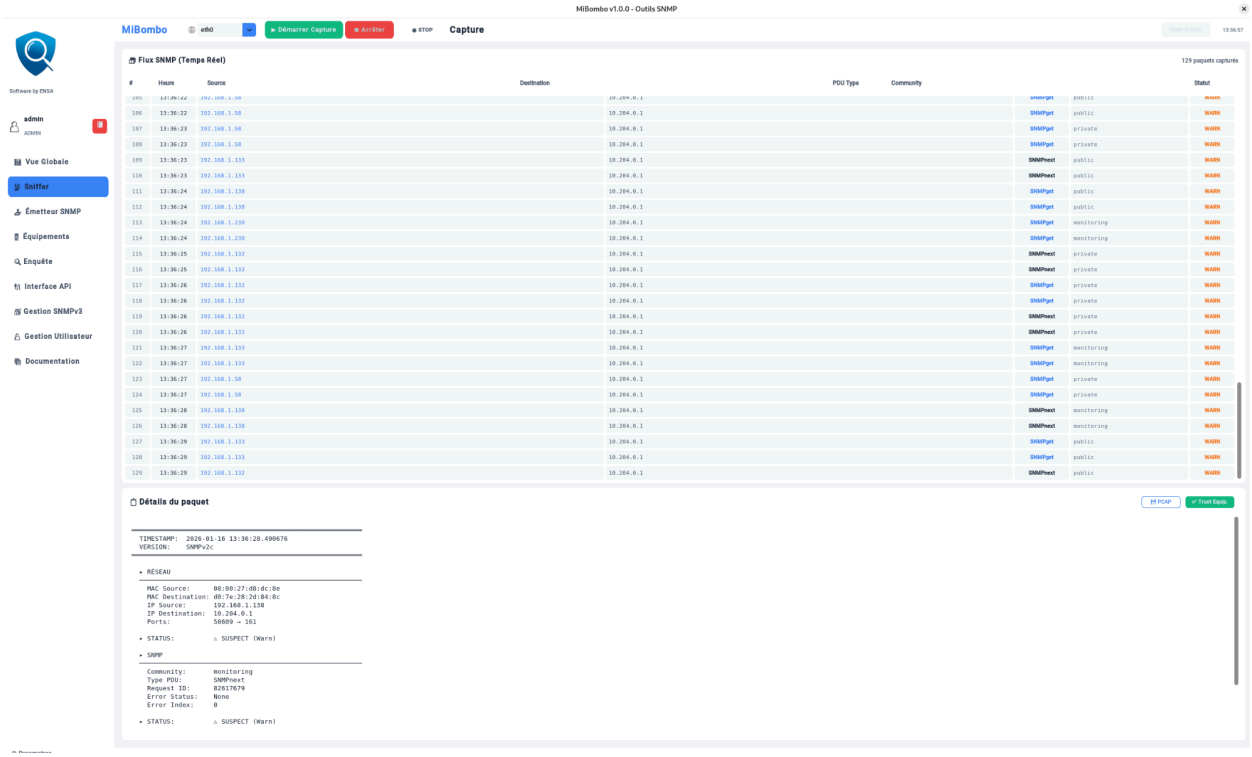
6.3 Onglet Sniffer Trames SNMP(GUI)

L'onglet Sniffer permet de capturer et visualiser en temps réel les paquets SNMP transitant sur le réseau.

L'interface affiche une liste chronologique de tous les paquets capturés avec les informations essentielles : horodatage, adresses IP source/destination, version SNMP, type de PDU et statut de conformité (🟢 OK / 🟡 WARN / 🔴 ALERT).

En cliquant sur une trame dans la liste, le panneau de détails s'affiche en bas de l'écran. Il présente l'ensemble des informations du paquet sélectionné : adresses MAC, ports UDP, community string (v1/v2c) ou paramètres USM (v3), liste des OID interrogés avec leurs valeurs, et pour SNMPv3, le statut du déchiffrement si les credentials sont configurés.

Des boutons d'action permettent de faire confiance à une source (whitelist), de la bloquer, ou d'exporter le paquet.



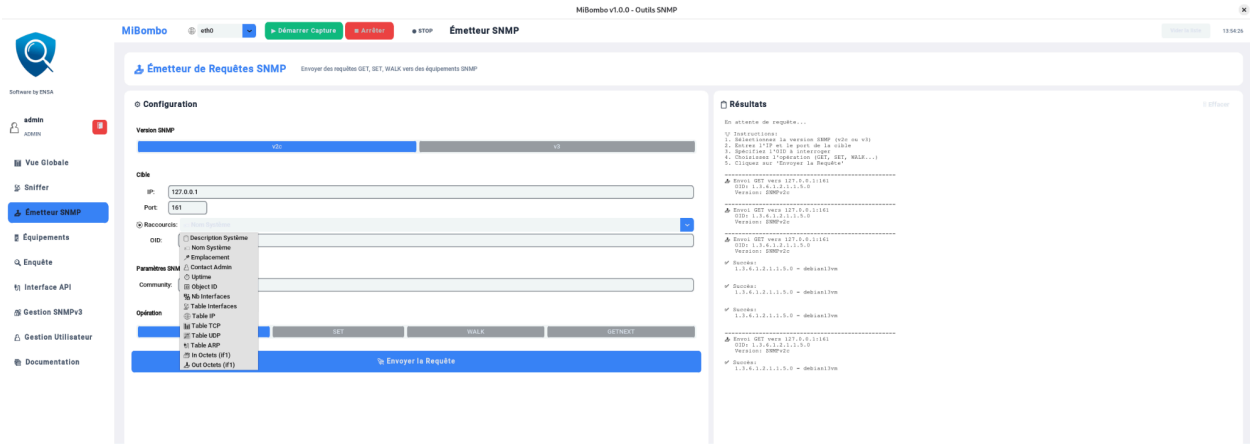
cf. Figure 27 : Onglet Sniffer avec liste des paquets et détails d'une trame sélectionnée

6.3 Onglet Emetteur SNMP

L'onglet Émetteur SNMP permet d'envoyer des requêtes SNMP (GET, SET, WALK, GETNEXT) vers des équipements du réseau pour tester leur connectivité ou interroger des OID spécifiques.

L'utilisateur configure la cible (IP, port), sélectionne la version SNMP (v2c ou v3) avec les paramètres d'authentification correspondants, puis choisit l'OID à interroger parmi une liste de raccourcis prédéfinis (Description Système, Uptime, Table Interfaces, etc.) ou en le saisissant manuellement.

Les résultats s'affichent dans le panneau de droite avec le temps de réponse et les valeurs retournées.



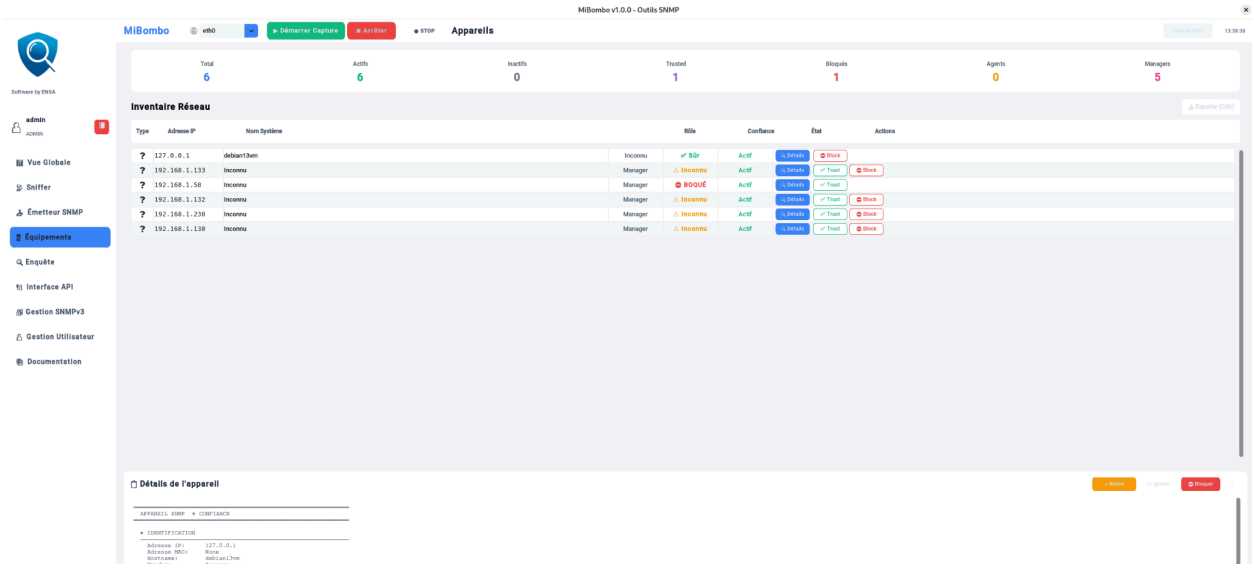
*cf. Figure 28 : Interface de l'émetteur SNMP avec configuration et résultats**

6.3 Onglet Emetteur SNMP

L'onglet Équipements présente la liste de tous les appareils détectés sur le réseau lors des captures SNMP.

Pour chaque équipement, l'interface affiche : l'adresse IP, l'adresse MAC, le type d'appareil détecté, le fabricant (vendor), la date de première et dernière observation, ainsi que le nombre de paquets échangés.

Un système de statut permet de marquer les équipements comme "Whitelistés" (de confiance) ou "Blacklistés" (bloqués). Ces marquages influencent directement le niveau d'alerte affiché dans l'onglet Sniffer pour les paquets provenant de ces sources.



cf. Figure 29 : Liste des équipements découverts avec leurs informations

6.4 API REST

Points d'Accès Principaux

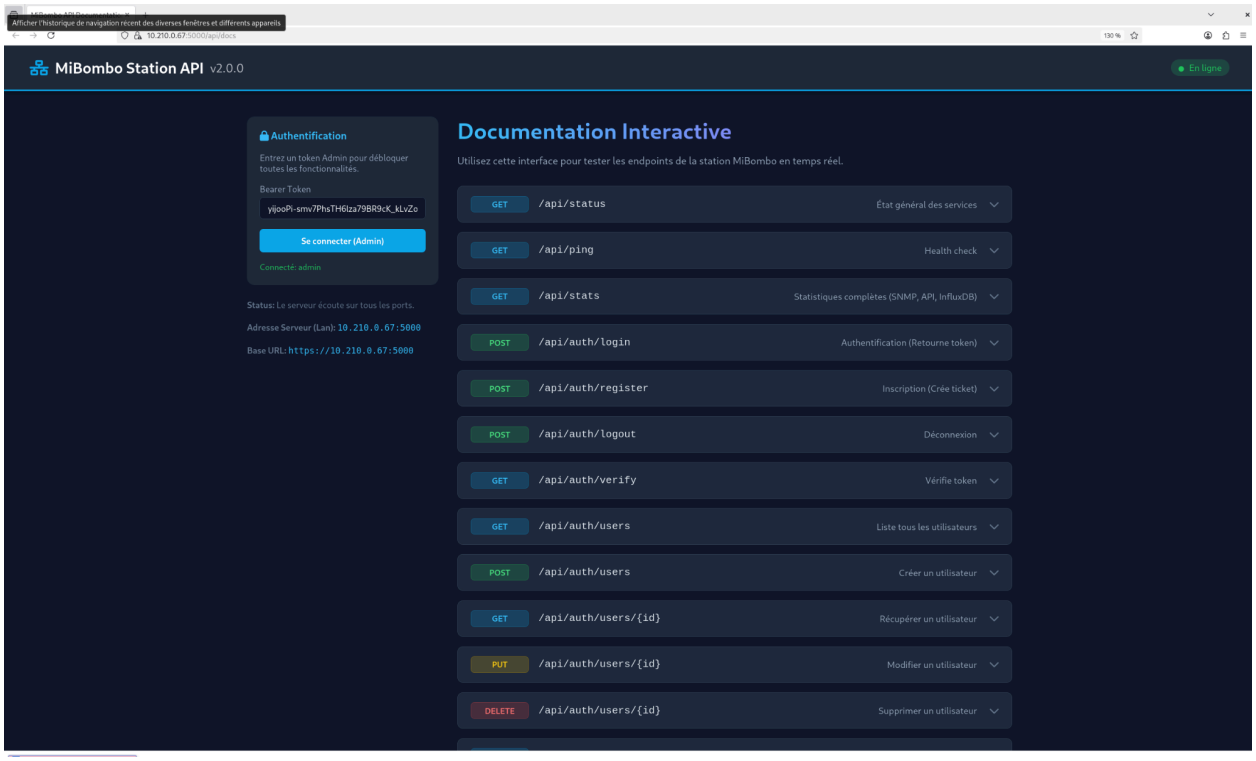
Méthode	Endpoint	Description
GET	`/api/status`	État de l'application
POST	`/api/capture/start`	Démarrer une capture
POST	`/api/capture/stop`	Arrêter la capture
GET	`/api/packets`	Liste des paquets capturés
GET	`/api/stats`	Statistiques globales

Méthode	Endpoint	Description
GET	/api/alerts	Alertes d'anomalies

Authentification API

L'API utilise des tokens Bearer ou Json Web Token (JWT) :

```
# Obtenir un token
curl -X POST https://adresseip:5000/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"username": "admin", "password": "admin"}' \
  --insecure

# Utiliser le token
curl https://adresseip:5000/api/status \
  -H "Authorization: Bearer <TOKEN>" \
  --insecure
```



cf. Figure 26 : Documentation interactive de l'API (Swagger UI)

6.5 Mode Ligne de Commande (CLI) - Version instable

Le mode CLI est utile pour : - Les serveurs sans interface graphique - L'intégration dans des scripts - Le déploiement headless (Docker, VM)

Exemple de Session CLI

```
$ sudo python main.py --cli -i eth0 --with-api

============================================================
     MiBombo V1.1.0 - Mode CLI
============================================================
  Interface: eth0
  Filtre: udp port 161 or udp port 162
  Database: mibombo.db
============================================================
[+] API REST: http://0.0.0.0:5000

[*] Capture en cours... (Ctrl+C pour arreter)

[LIVE] Paquets:    142 | Alertes:    3
```

cf. Figure 27 : Session CLI en cours

---

7. Fonctionnalités Détaillées

7.1 Capture de Paquets SNMP

Démarrage de la Capture

Depuis l'interface graphique : 1. Sélectionner l'interface réseau dans les paramètres 2. Cliquer sur le bouton "Démarrer la capture" 3. Observer les paquets apparaître en temps réel

cf. Figure 28 : Bouton de démarrage de capture

Filtre BPF

Le filtre par défaut est :

```
udp port 161 or udp port 162
```

Personnalisable pour cibler des hôtes spécifiques :

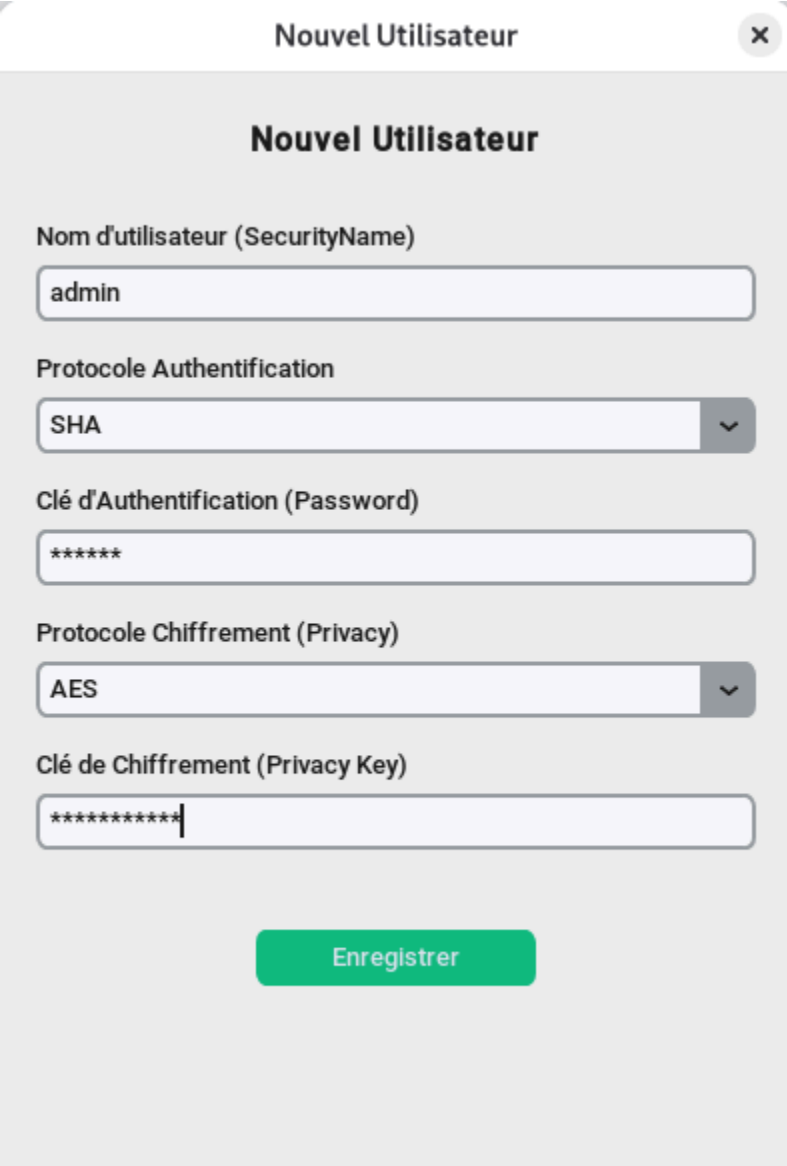
```
host 192.168.1.100 and (udp port 161 or udp port 162)
```

7.2 Analyse et Déchiffrement SNMPv3

MiBombo supporte le déchiffrement des paquets SNMPv3 sécurisés.

Prérequis

Pour déchiffrer les messages SNMPv3, vous devez enregistrer les credentials des agents : 1. Aller dans Paramètres > Credentials SNMPv3 2. Ajouter un utilisateur : Username, Auth Protocol (MD5/SHA), Auth Key, Priv Protocol (DES/AES), Priv Key



cf. Figure 29 : Configuration des credentials SNMPv3

Niveaux de Sécurité

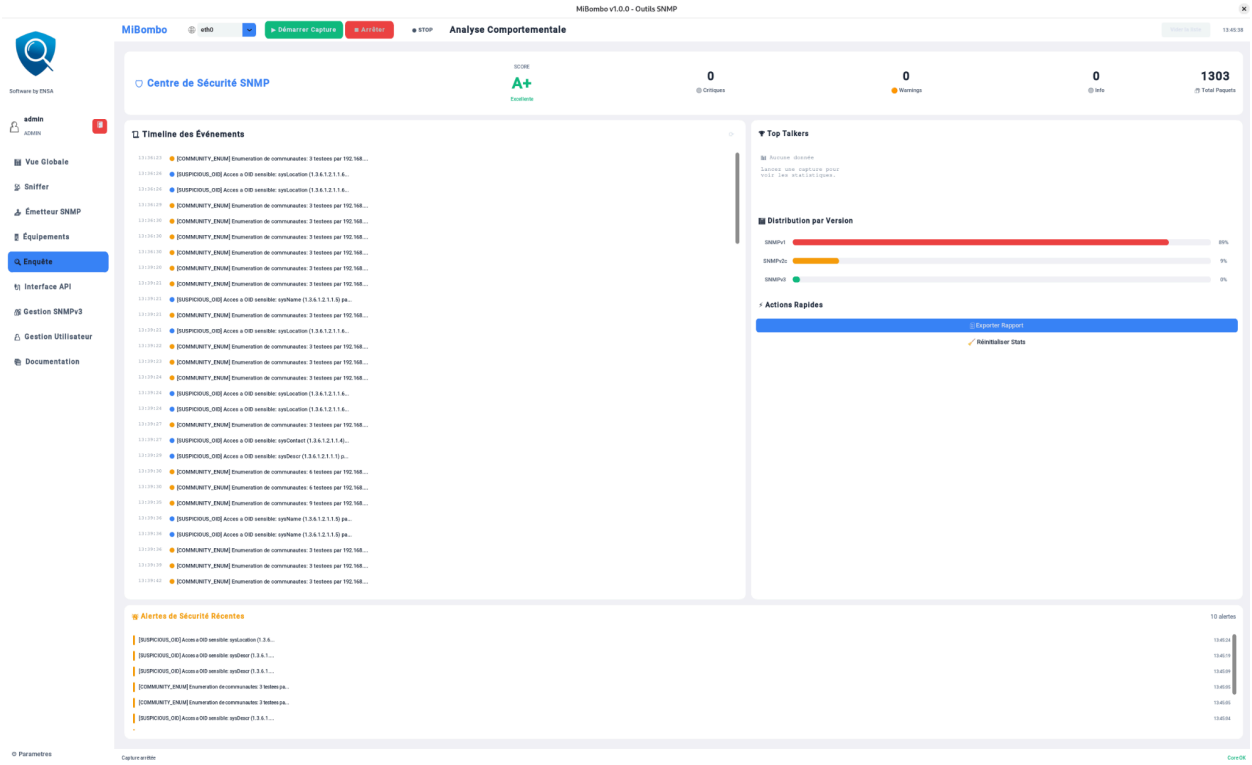
Niveau	Auth	Priv	Description
`noAuthNoPriv`	❌	❌	Aucune sécurité
`authNoPriv`	✅	❌	Authentification uniquement
`authPriv`	✅	✅	Authentification + Chiffrement

7.3 Détection d'Anomalies Enquête

Le module de détection analyse en permanence le trafic et génère des alertes.

Types d'Anomalies Détectées

Type	Description	Sévérité
Community Scan	Tentatives multiples avec différentes communautés	Warning
Traffic Spike	Augmentation soudaine du nombre de requêtes	Info
Unknown Source	Nouvelle IP non répertoriée	Info
Suspicious OID	Accès à des OID sensibles	Critical
Flood Attack	Volume anormal depuis une même source	Emergency
Hping	Spam de ping	Critical



cf. Figure 30 : Extrait du software de la detection d'anomalie

7.4 Gestion des Utilisateurs

Rôles et Permissions

Rôle	Permissions
admin	Toutes les permissions (gestion utilisateurs, configuration)
operator	Capture, analyse, export des données
viewer	Consultation seule (lecture des statistiques)

Administration des Comptes

Accessible via Paramètres > Gestion des Utilisateurs : - Valider les demandes d'inscription - Activer/Désactiver des comptes - Réinitialiser les mots de passe (a implémenter)

8. Sécurité

8.1 Authentification Sécurisée

Stockage des Mots de Passe

Les mots de passe sont hashés avec **Argon2id**, l'algorithme recommandé par OWASP : - Résistant aux attaques GPU - Protection contre les attaques par canal auxiliaire - Salage automatique

Protection Contre les Attaques Reconnus

Attaque	Protection
Brute Force	Limitation du nombre de tentatives (rate limiting)
Session Hijacking	Tokens JWT avec expiration courte
CSRF	Tokens CSRF sur les formulaires
XSS	Échappement des entrées utilisateur

8.2 Gestion des Certificats

Renouvellement Automatique

Configurer un cron avez crontab pour le renouvellement automatique :

```
# Renouveler le certificat Station tous les 80 jours
0 2 */80 * * /opt/mibombo/scripts/pki/renew_station_cert.sh && systemctl
restart mibombo
```

Vérification des Certificats

```
# Vérifier l'expiration
openssl x509 -in pki/station/station.pem -noout -dates
```

8.3 Bonnes Pratiques

1. **Ne jamais utiliser les identifiants par défaut** en production
2. **Choisissez un mail sur et personnel**
3. **Sauvegarder régulièrement** la base de données et les configurations
4. **Mettre à jour** les dépendances Python régulièrement
5. **Isoler** le serveur MiBombo dans une machine sécruisé si possible

9. Maintenance et Dépannage

9.1 Logs et Diagnostics

Emplacement des Logs

```
data/logs/
├── mibombo.log         # Log principal de l'application
├── api.log             # Log des requêtes API
├── security.log        # Événements de sécurité (connexions, alertes)
└── sniffer.log         # Log du module de capture
```

Niveaux de Log

Niveau	Description
`DEBUG`	Informations de débogage détaillées
`INFO`	Événements normaux
`WARNING`	Situations inhabituelles non bloquantes
`ERROR`	Erreurs nécessitant une attention
`CRITICAL`	Erreurs graves empêchant le fonctionnement

cf. Figure 34 : Exemple de fichier de log

9.2 Problèmes Courants

Erreur : "SNIFFER_KEY manquante"

Cause : Le fichier `.env` n'est pas configuré ou la clé est invalide.

Solution :

```
cp .env.example .env
python3 -c "from cryptography.fernet import Fernet; print('SNIFFER_KEY=' +
Fernet.generate_key().decode())" >> .env
```

Erreur : "Permission denied" lors de la capture

Cause : L'application n'a pas les droits root.

Solution :

```
sudo ./run.sh
```

NB : dans le prochain patch une version cliquable sera msie en place

Erreur : “No module named ‘xyz’”

Cause : Dépendance Python manquante.

Solution :

```
pip install -r requirements.txt
```

L’interface ne s’affiche pas (environnement headless)

Cause : Aucun display X11 disponible.

Solution : Utiliser le mode CLI ou configurer X11 forwarding :

```
ssh -X user@serveur
sudo python main.py
```

9.3 Sauvegarde et Restauration

Sauvegarde Complète

```
# Sauvegarde des données et configurations
tar -czf mibombo_backup_$(date +%Y%m%d).tar.gz \
    --exclude=venv --exclude=__pycache__ \
    mibombo.db config/ data/ .env
```

Restauration

```
tar -xzf mibombo_backup_20260116.tar.gz -C /opt/mibombo/
```

cf. Figure 36 : Processus de sauvegarde et restauration

---

10. Annexes

10.1 Arborescence du Projet

```
MiBombo_version1/
├── api/
│   ├── __init__.py
│   └── api.py                  # Définition de l'API REST Flask
├── assets/
│   └── logo.png                # Logo de l'application
├── core/
│   ├── __init__.py
│   ├── analyzer.py             # Analyse des paquets SNMP
│   ├── anomaly_detector.py     # Détection d'anomalies
│   ├── app_config.py           # Configuration applicative
│   ├── authentication.py       # Authentification legacy
│   ├── logger.py               # Système de logging
│   ├── mailer.py               # Envoi d'e-mails (2FA)
│   ├── PostgresDB.py           # Couche d'accès base de données
│   ├── secure_authentication.py # Auth sécurisée (nouveau)
│   ├── sniffer.py              # Capture de paquets
│   ├── snmp_credentials.py     # Gestion des credentials SNMPv3
│   └── ssl_config.py           # Configuration SSL/TLS
├── gui/
│   ├── __init__.py
│   ├── auth_panel.py           # Widgets d'authentification
│   ├── documentation_tab.py    # Onglet documentation
│   ├── extensions.py           # Extensions GUI
│   ├── legacy_auth_widgets.py  # Widgets auth legacy
│   ├── main_gui.py             # Interface principale
│   └── sem_topology.py         # Widget topologie
├── scripts/
│   └── pki/
│       ├── generate_pki.sh     # Génération PKI complète
│       └── renew_station_cert.sh # Renouvellement certificat
├── tests/
│   ├── test_api.py
│   ├── test_auth.py
│   └── test_components.py
├── utils/
│   ├── __init__.py
│   └── db_viewer.py            # Visualiseur base de données
├── .env.example                # Template variables d'environnement
├── docker-compose.yml          # Configuration Docker
├── Dockerfile                  # Image Docker
├── main.py                     # Point d'entrée
├── requirements.txt            # Dépendances Python
└── run.sh                      # Script de lancement
```

10.3 Glossaire

Terme	Définition
API	Application Programming Interface – Interface de programmation
ASN.1	Abstract Syntax Notation One – Notation de syntaxe abstraite utilisée par SNMP
BER	Basic Encoding Rules – Règles d'encodage de base pour ASN.1
BPF	Berkeley Packet Filter – Filtre de paquets utilisé pour la capture réseau
CA	Certificate Authority – Autorité de certification
CLI	Command Line Interface – Interface en ligne de commande
CSRF	Cross-Site Request Forgery – Falsification de requête inter-sites
DES	Data Encryption Standard – Algorithme de chiffrement symétrique
FQDN	Fully Qualified Domain Name – Nom de domaine pleinement qualifié
GUI	Graphical User Interface – Interface graphique utilisateur
JWT	JSON Web Token – Jeton d'authentification au format JSON
MIB	Management Information Base – Base d'informations de gestion SNMP
NOC	Network Operations Center – Centre d'opérations réseau
OID	Object Identifier – Identifiant d'objet dans l'arborescence SNMP
PCAP	Packet Capture – Format de fichier de capture réseau
PDU	Protocol Data Unit – Unité de données du protocole SNMP
PKI	Public Key Infrastructure – Infrastructure à clé publique
REST	Representational State Transfer – Architecture de services web

Terme	Définition
SNMP	Simple Network Management Protocol – Protocole de gestion réseau
SOC	Security Operations Center – Centre d'opérations de sécurité
SSL/TLS	Secure Sockets Layer / Transport Layer Security – Protocoles de sécurisation
USM	User-based Security Model – Modèle de sécurité utilisateur de SNMPv3
XSS	Cross-Site Scripting – Injection de scripts inter-sites

10.4 Références

Standards et RFCs

Document	Titre	Lien
RFC 3411	Architecture for SNMP	https://datatracker.ietf.org/doc/html/rfc3411
RFC 3412	Message Processing and Dispatching	https://datatracker.ietf.org/doc/html/rfc3412
RFC 3414	User-based Security Model (USM)	https://datatracker.ietf.org/doc/html/rfc3414
RFC 3416	Protocol Operations for SNMPv2	https://datatracker.ietf.org/doc/html/rfc3416
RFC 3418	MIB for SNMP	https://datatracker.ietf.org/doc/html/rfc3418

Documentation des Technologies

Technologie	Documentation
Python 3.11	https://docs.python.org/3.11/
Scapy	https://scapy.readthedocs.io/
CustomTkinter	https://customtkinter.tomschimansky.com/
Flask	https://flask.palletsprojects.com/
PostgreSQL	https://www.postgresql.org/docs/
Docker	https://docs.docker.com/
Cryptography	https://cryptography.io/en/latest/

Ressources de Sécurité

Ressource	Lien
OWASP Authentication Cheat Sheet	https://cheatsheetseries.owasp.org/
CIS Benchmarks	https://www.cisecurity.org/cis-benchmarks
ANSSI Guides	https://www.ssi.gouv.fr/guide/

---

11. Perspectives d'Évolution

Ce chapitre présente la feuille de route technique et les améliorations planifiées pour les futures versions de MiBombo.

11.1 Roadmap Technique

```
Q1 2026 (Actuel)                Q2 2026
──────────────────────          ──────────────

✅ v1.1.0                       🔄 v1.2.0                 📋 v2.0.0
• SNMPv3 complet                • Résolution DNS          • Interface web
• Détection anomalies           • Dashboard avancé        • Multi-instances
• API REST                      • Alertes email           • Machine Learning
• Auth SMTP                                               • Clustering HA
```

cf. Figure 37 : Roadmap des versions MiBombo

🔧 Améliorations Techniques Continues

Domaine	Amélioration
Performance	Optimisation du parsing SNMP, réduction de l'empreinte mémoire
Sécurité	Audit régulier des dépendances, tests de pénétration
Tests	Couverture de test > 80%, tests d'intégration automatisés
Documentation	Tutoriels vidéo, exemples d'intégration API
Accessibilité	Conformité WCAG 2.1 pour l'interface graphique

11.3 Contributions et Retours

Comment Contribuer

MiBombo est un projet développé dans un cadre académique. Les contributions et retours sont les bienvenus :

1. **Signaler un bug** : Ouvrir une issue sur le dépôt GitHub
2. **Proposer une fonctionnalité** : Décrire le besoin dans une issue dédiée
3. **Contribuer au code** : Fork, développement, Pull Request
4. **Documentation** : Améliorer ou traduire la documentation existante

Contact

Canal	Utilisation
GitHub Issues	Bugs, demandes de fonctionnalités
Email projet	Contact direct avec l'équipe de développement
IUT de Béziers	Encadrement académique

Remerciements

L'équipe de développement remercie :

- **M. Druon Sébastien**, **M. Borelly Christophe** et **M. Comby Frédéric** pour leur encadrement et leurs conseils ainsi que leurs écoute et leurs exigence en tant que client pour ENSA
- L'ensemble des enseignants du département **R&T de l'IUT de Béziers**
- La communauté open source pour les bibliothèques utilisées

Document rédirigé le 14 Janvier 2026

MiBombo v1.1.0

© 2026 - IUT de Béziers - BUT R&T